



Assignment: Distributed Banking System

by

Name	Matric No.	Work %
Timothy Chang	<i>U2220136J</i>	33.33%
O Jing	<i>UXXXXXXX</i>	33.33%
Neo Zhi Xuan	<i>U2222293E</i>	33.33%

SC4051 Distributed Systems

NANYANG TECHNOLOGICAL UNIVERSITY

April 2026

Contents

1 Introduction & System Overview

2 Message Design & Wire Protocol

- 2.1 Packet Header Format
- 2.2 Type-Length-Value (TLV) Payload Encoding
- 2.3 Callback Push Packet
- 2.4 Cross-Language Marshalling Challenges

3 System Architecture & Design

- 3.1 Server Architecture (Go)
- 3.2 Client Architecture (C++)
- 3.3 Concurrency Model for Accounts

4 Banking Services & Custom Operations

- 4.1 Core Services (S1–S4): Open, Close, Deposit, Withdraw
- 4.2 Monitor Service (S5)
- 4.3 GetBalance (Idempotent Operation)
- 4.4 Transfer (Non-Idempotent Operation)

5 Fault Tolerance & Invocation Semantics

- 5.1 At-Least-Once Semantics
- 5.2 At-Most-Once Semantics
- 5.3 Loss Simulator Design
- 5.4 Experimental Validation
- 5.5 Results & Discussion

6 Testing Strategy

- 6.1 Unit Tests (C++ Client)
- 6.2 Integration Tests (End-to-End)

7 Assumptions, Limitations & Future Work

- 7.1 System Assumptions
- 7.2 Known Limitations
- 7.3 Future Work
- 7.4 Key Takeaways

8 Conclusion

1 Introduction & System Overview

The project implements a distributed banking application, using a custom client-server architecture that uses UDP-based inter-process communication. The system is cross-language interoperable and consists of a C++ client application and a Go-based server. These components communicate via a custom binary protocol, without Remote Procedure Call (RPC) frameworks.

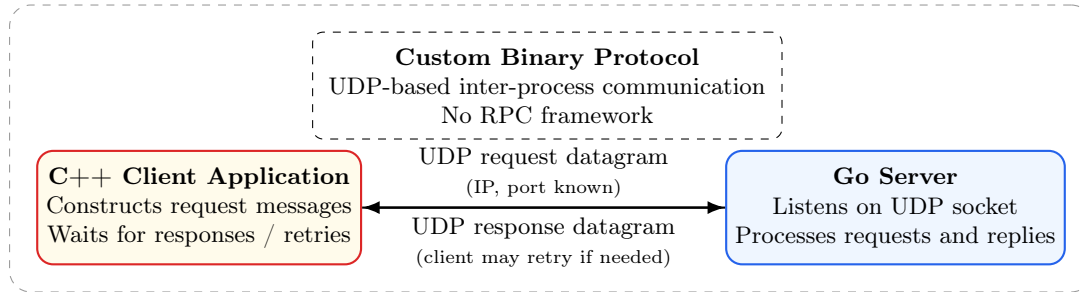


Figure 1: High-level architecture of the distributed banking system.

At a high level, the client and server communicate via UDP datagrams, on known IP addresses and ports. The client constructs request messages and waits for responses, and may retry depending on the protocol. The server listens for requests and processes each one, before returning a response.

The client is written in C++ due to the language’s ability to handle low level byte control for packet construction, as well as portable socket handling across Windows and POSIX. Meanwhile, the choice of Go for the server provides an in-depth networking library and built-in synchronisation primitives for concurrency, to allow for simple implementation of UDP sockets and concurrent client handling.

The presence of cross-language components necessitates handling of various constraints. Firstly, the data boundary and marshalling conventions for all data types must be aligned on both components. Next, it is important to standardise encodings so that both components interpret numeric bytes identically. Network byte order (big-endian) is used for all integer fields, whereas IEEE754 double format (64-bit) is used for floating-point balances.

The report analyses the unreliability of UDP, and the trade-offs between using different semantic modes to handle it, via explicit retry and duplicate-detection mechanisms.

2 Message Design & Wire Protocol

2.1 Packet Header Format

Each message sent between client and server begins with an 18-byte header, followed by a payload of variable length. It encodes information about protocol control, through these fields:

Table 1: Packet Header Layout

Field	Size	Description
Type	1 byte	Indicates whether the datagram is a request (0x00), a normal reply (0x01), or a server-initiated callback (0x02).
Semantics Flag	1 byte	Encodes invocation semantics: 0x00 for at-least-once, 0x01 for at-most-once.
Request ID	4 bytes	An identifier chosen by the client, used for duplicate detection and request-response matching.
Client IPv4 Address	4 bytes	The sender’s IPv4 address. Explicitly informs the server about the client’s identity.
Client Port	2 bytes	The UDP port number of the sender.
Status Code	2 bytes	Indicates success or error conditions in replies; non-zero values represent errors.
Content Length	4 bytes	Length (in bytes) of the payload that follows.

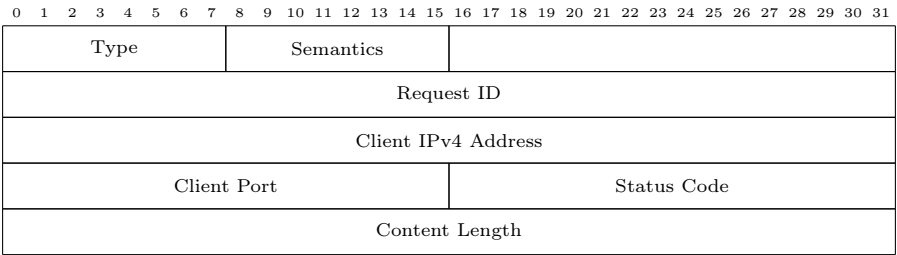


Figure 2: Byte-level representation of the packet header.

This header is inspired by traditional RPC or network protocol headers. A fixed header size allows the client and server to parse data with a consistent offset. It also allows for robust validation of datagrams that are truncated. Simultaneously, all multi-byte fields are in big-endian format, aligning the client and server’s method of interpreting numeric values.

2.2 Type-Length-Value (TLV) Payload Encoding

After the header, each message carries a TLV-encoded payload of application-specific fields. Fields are optional, and only serialised when it is present. Each field has this format:

Table 2: TLV Payload Layout

Field	Size	Description
Field ID	1 byte	Identifies the type of the field (e.g., account number, password, amount).
Length	4 bytes	Specifies the length of the Value field in bytes, encoded in big-endian format.
Value	Variable	Contains the actual data corresponding to the Field ID. The size is determined by the Length field.

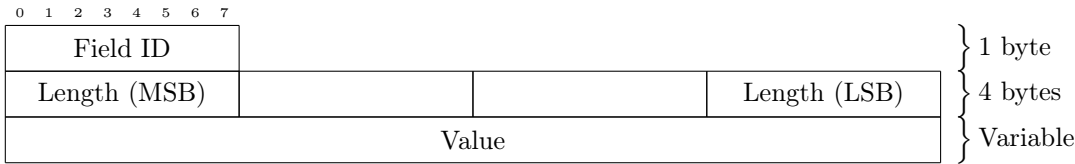


Figure 3: Type-Length-Value (TLV) encoding used in protocol payloads.

For example, the request "OpenAccount (Alice, pass1234, SGD, 1000.00)" attempts to open an account, and might include these TLV fields:

Table 3: OpenAccount Request Fields

Field Name	Length	Description
Account Holder Name	N bytes	UTF-8 encoded string representing the user’s name. The length varies depending on the number of characters.
Password	Up to 8 bytes	Password represented as raw bytes. If shorter than 8 bytes, it is zero-padded to maintain a fixed maximum length.
Currency Type	1 byte	Encoded as a predefined enumeration value representing supported currencies (e.g., SGD, USD).
Initial Balance	8 bytes	IEEE-754 double precision floating-point number encoded in big-endian format.

The client packs each numeric or string value without additional padding, and the length specifies the exact number of bytes. For example, if the user “Alice” (5 letters) chooses password “pass1234” (8 chars), currency SGD (id=1), and balance 1000.00, the TLV encoding for that request would look like:

Listing 1: TLV Encoding Example

10 00 00 00 01 01

[ServiceID field: ID=16, Len=1, Value=0x01 (

OpenAccount service)]

```

11 00 00 00 01 01          [MsgType field: ID=17, Len=1, Value=0x01 (Request)
]
01 00 00 00 05 41 6C 69 63 65 [Name field: ID=1, Len=5, Value="Alice"]
02 00 00 00 08 70 61 73 73 31 32 33 34 [Password field: ID=2, Len=8, Value="pass1234"]
03 00 00 00 01 01          [Currency field: ID=3, Len=1, Value=0x01 (SGD)]
04 00 00 00 08 40 8F 40 00 00 00 00 00 [Balance field: ID=4, Len=8, Value=1000.00]

```

The server, upon receiving this TLV payload, unpacks each field by reading the ID and length, then extracts the number of bytes as the value. It interprets the raw bytes based on the field's type - converting an 8-byte double to Go's float64 in big-endian, or treating password bytes as an ASCII string.

Meanwhile, certain fields are fixed-size. In the Go server, an 8-byte password field is enforced. This will be discussed in detail in Section 2.4.

Hence, this format handles data cleanly. The use of ID and length allows for robust parsing. While the encoding is handcrafted in code, it creates an effective platform-independent message format.

2.3 Callback Push Packet

Within the lab requirements, the server should "push" updates to all registered clients during a monitoring interval. When an account is opened, closed, deposited, or withdrawn, the server triggers a callback packet to the client. It uses a different packet format:

Table 4: Callback Packet Layout

Field Name	Size	Description
MsgType	1 byte	Always set to 0x02 to differentiate callback packets from normal replies.
Service ID	1 byte	Indicates the operation to trigger (e.g., OpenAccount).
Account Number	4 bytes	Integer account number (big-endian).
Name Length	4 bytes	Length of the account holder's name in bytes.
Name	Variable	UTF-8 encoded string representing the account holder's name.
Currency Type	1 byte	The account's currency code as an enumeration.
Balance	8 bytes	IEEE-754 double precision floating-point value (big-endian) representing the updated balance.

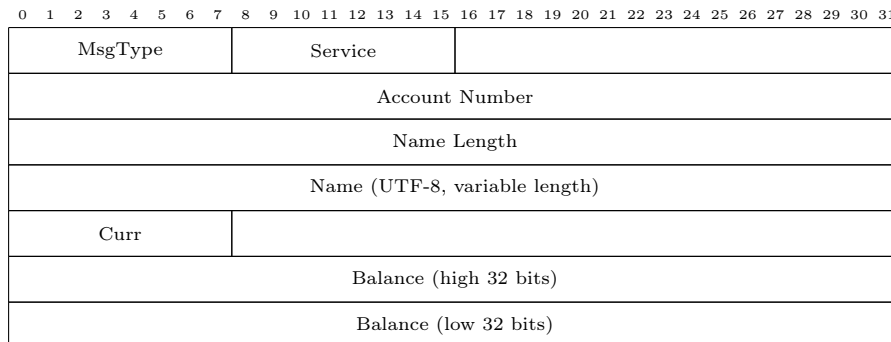


Figure 4: Byte-level representation of the callback payload.

For example, if account 12345 ("Bob") with currency USD (0x00) now has balance 250.75, the server might send:

Listing 2: Callback Packet Example

```

00 00 30 39          [Account#=12345 (big-endian)]
00 00 00 03 42 6F 62 [NameLen=3, Name="Bob"]
00                  [Currency=0x00 (USD)]
40 6F 40 00 00 00 00 00 [Balance=250.75 (IEEE-754)]

```

The client's UDP socket listens for these packets and, upon receipt, displays the updated account information to the user.

The benefits of having a fixed, known structure is that it avoids the overhead that TLV packets face with self-describing fields. TLV packets require 5 additional bytes for each field to store FieldID and Length,

making it slower to encode, decode, and process.

2.4 Cross-Language Marshalling Challenges

Implementing the client and server in different languages always leads to challenges in data coordination, due to the differences in how each language defines its data types and mechanisms. The first challenge was designing the password field. The Go server defines passwords as a fixed 8-byte array, whereas the C++ client uses ‘std::string’. To overcome this, the protocol’s TLV password field was fixed to 8 bytes on the wire. The client would ensure that the string is 8 bytes, by truncating it, or padding it with zeros.

The second issue is the different representation of floating-point values in memory across both languages, where C++ uses ‘double’ and Go uses ‘float64’. While both languages use IEEE-754 64-bit binary representation for these numbers, they could differ in endianness, and lead to incorrect decoding of raw bytes. Hence, for all floating-point transmissions, values are converted into IEEE-754 format encoded in big-endian order before sending, and recipients would reverse the process. This ensures the same byte sequence is interpreted reliably and correctly.

3 System Architecture & Design

3.1 Server Architecture (Go)

The Go server is implemented with a layered design.



Figure 5: Go server architecture (high-level).

Firstly, main function sets up a UDP socket and constructs the server components.

Next, the dispatcher reads incoming UDP packets, unmarshals the payload and header, before routing each request to an appropriate service handler. Its behaviour depends on the chosen invocation semantics: for at-least-once mode, it forwards each request to the handler, while in at-most-once mode, it checks the request against a history table to eliminate duplicate requests.

Thirdly, the handler receives a request from the dispatcher, and chooses a service to trigger. It contains the application logic for services like opening, closing, depositing and withdrawing from an account. It decodes the TLV fields from the payload and performs validation, before invoking the appropriate service function, and encoding a TLV response.

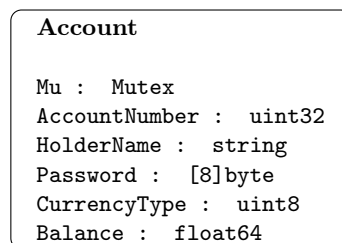


Figure 6: Memory store account structure.

Lastly, the banking service and memory store layers handle the data operations. The service layer provides operations like OpenAccount that directly manipulate the storage. The memory store maintains a map of accounts, and uses synchronisation primitives to ensure that concurrent access to data is safe and does not lead to race conditions.

Separately, the monitoring feature is implemented within the banking service with a publish-subscribe mechanism. It stores a map of registered clients with their defined expiration times, and handles the goroutines for dispatching account updates using callback packets.

By separating the logic into layers, we isolate concerns cleanly. For instance, semantics logic is handled in the dispatcher, and does not clutter banking service logic. Protocol encoding is handled only once in the dispatcher and handler. At the same time, this allows dependency injection in tests, by allowing developers to mock the UDP socket at the top level.

The architecture is wired by a BuildHandler function that exposes the banking service and monitor manager

to all handlers. This ensures that the dispatcher can make calls to the handler without having to know the underlying banking or monitoring details, hence decoupling these components.

3.2 Client Architecture (C++)

The C++ client is implemented with a three-layer architecture.



Figure 7: C++ client layered architecture.

When sending a request from the application layer, the client runs an interactive looping Command Line Interface (CLI) that takes a command from the user corresponding to banking operations. It communicates this request to the protocol layer. The protocol layer constructs a request message by populating the 18-byte header with the chosen invocation semantics flag and a fresh or zeroed request ID. It then converts numbers into big-endian format before encoding arguments into a TLV payload for cross-platform consistency. This layer also manages the decoding of replies and callbacks into structured data. Finally, the network layer abstracts UDP communication by providing send and receive operations over sockets.

When the client receives a response, the response takes the reverse path. It is received at the network layer. Then, the protocol layer validates headers and parses the payload, and thereafter the application layer presents the results or callback updates to the user. This design ensures that user logic, message formatting, and network communication are cleanly decoupled and easy to maintain.

At the same time, dependency injection was used to provide the client with access to various services. The client was able to access interfaces like BaseSocket and BaseCommandEncoder during its lifecycle. This allows the system to mock components for unit testing, and reduces coupling between the interface and networking/marshalling internals.

3.3 Concurrency Model for Accounts

While the lab permits a single-threaded server, our system was designed to allow multiple clients to operate concurrently on the server, and requires various synchronisation strategies to align this.

The in-memory account map, where account numbers were mapped to account data, is protected by a global read-write mutex which synchronises account lifecycle operations like CreateAccount and GetAccount. While multiple read queries can be run under a shared read lock, write-based operations utilise an exclusive write lock.

Each account's state is protected separately using mutexes that belongs to the account itself. After accessing the specific account from the store, the account's mutex is acquired before read or mutation operations, and only unlocked when completed. This design allows clients to concurrently operate on different accounts while ensuring correctness on individual accounts.

4 Banking Services & Custom Operations

4.1 Core Services (S1–S4): Open, Close, Deposit, Withdraw

The server implements four core banking services:

OpenAccount (S1): Creates a new account with an initial balance, currency, and password. The server generates a unique account number, and stores the account under the global store write lock. This service returns the generated account number.

CloseAccount (S2): Deletes an account after verifying credentials. The handler retrieves the account, validates the user's credentials, locks the account to prevent concurrent mutations, and removes it from the store. Notifications are sent to monitor subscribers.

Deposit (S3): Adds funds to an account. After credential validation, the handler acquires the account's per-account mutex, checks that currency matches, increments the balance, and returns the new balance. The transaction is atomic with respect to other operations on the same account.

Withdraw (S4): Removes funds from an account. Similar to deposit, but additionally checks that sufficient balance exists. If any check fails (wrong credentials, currency mismatch, insufficient funds), an error status is returned without modifying state.

All modifying operations above trigger monitor callbacks after state changes. Errors are reported via the reply header's status code, allowing clients to distinguish between errors like authentication failures and missing accounts.

4.2 Monitor Service (S5)

The monitor feature allows clients to subscribe to real-time account update notifications. When a client issues a monitor request, specifying a timeout interval, the server registers the client's address and expiration time within a mutex-protected subscriber map. A background goroutine periodically sweeps this map, removing expired entries.

Whenever a modifying operation (S1-S4) occurs, the handler invokes `NotifyAll()`, which marshals the account update once, then iterates through active subscribers (under lock) and sends a callback packet to each. Expired subscribers are lazily evicted during iteration. This design avoids holding account locks during network I/O.

On the client side, after receiving a monitor acknowledgment, the application enters a blocking receive loop on the UDP socket, collecting callbacks until the timeout expires. User input is disabled during this period.

4.3 GetBalance (Idempotent Operation)

`GetBalance` queries an account's current balance without modifying state. After credential and currency validation, the server reads the balance under the account's mutex and replies with this value. Because the operation is read-only, it is inherently *idempotent*: repeating the request yields the same result without side effects. This property is valuable in UDP networks: if a reply is lost and the client retries, the outcome remains correct. Under both at-least-once and at-most-once semantics, idempotent operations are safe to retry, simplifying the reliability mechanisms.

4.4 Transfer (Non-Idempotent Operation)

`Transfer` moves funds atomically between two accounts. The request specifies source and destination account numbers, source credentials, amount, and currency. The handler's implementation must prevent two key issues: deadlock and inconsistency.

Deadlock avoidance: Transfer operations involve two accounts and face the risk of a deadlock, if they hold the resource that the other party requires, and wait indefinitely for it to be free. To avoid this, an ordering is imposed on account locks: the account with the smallest account number is locked first, then the other account is locked. This deterministic order ensures that no two simultaneous transfers will lock the same two accounts in different orders.

Atomic mutation: Under the locks, the handler verifies that both accounts exist, credentials are valid, currency matches, and the source has sufficient funds. If any check fails, both locks are released and an error is returned. If valid, the handler subtracts from the source and adds to the destination atomically, then sends callbacks to monitor subscribers.

As `Transfer` modifies state (specifically, moves funds), it is *non-idempotent*: applying it twice would transfer funds twice, which is incorrect. This makes `Transfer` a critical case for duplicate suppression. Under at-least-once semantics, retransmitted requests would execute twice, doubling the transfer. Under at-most-once semantics, duplicate detection prevents re-execution, ensuring each transfer happens exactly once or not at all. This contrast motivates the need for correct invocation semantics, which is addressed in Section 5.

5 Fault Tolerance & Invocation Semantics

UDP is fundamentally an unreliable protocol, where packets could be lost, reordered, or duplicated. In this section, we cover two semantic models that tackle these issues, analyse their tradeoffs, and empirically validate them via a loss simulator.

5.1 At-Least-Once Semantics

In this model, during a failed transmission, the client retransmits requests with the same request ID, until a response arrives. The server executes each request it receives without filtering or caching of results.

Client-side mechanism: After encoding and serialising a request, the client sends it via UDP, and awaits a reply with a 3-second timeout. If no reply arrives within the timeout, the client retransmits with an

exponential backoff, doubling the timeout period. This occurs up to a maximum of 3 attempts.

Server-side mechanism: The dispatcher receives the request header and immediately dispatches it to the handler for execution, without filtering for duplication. All requests that are received are processed regardless of whether a similar request had previously arrived.

In the event that the request is lost, and the server never received it, when the client recovers and retransmits within 3 attempts, the server will process the request at least once. However, if the retransmission by the client is caused by a lost reply, the server may execute the same operation multiple times.

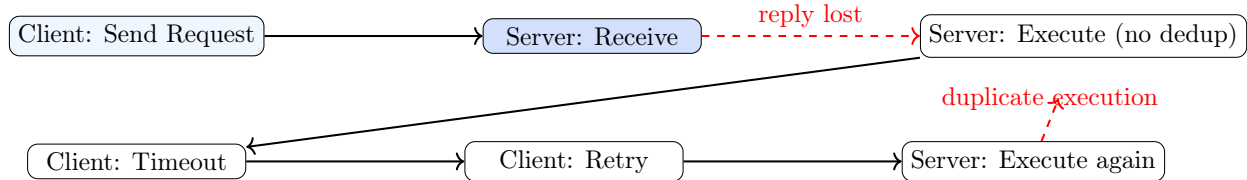


Figure 8: At-least-once semantics: retransmission on timeout can cause duplicate execution.

Therefore, at-least-once is safe only for idempotent operations like GetBalance, which can be repeated and still yield the same result. However, non-idempotent operations like Transfer are not suitable for this, as duplicate executions can lead to incorrect behaviour. For instance, a duplicate transfer would unexpectedly move funds twice, making this semantic unsuitable for banking operations without additional protections.

5.2 At-Most-Once Semantics

In this model, the client has a similar retry mechanism as at-least-once mode. However, the server detects and suppresses duplicates by caching replies, ensuring that each operation is executed at most one time, or zero times if all attempts are lost before execution.

Client-side mechanism: While the retry mechanism is similar to at-least-once, in this model, the client increments the request ID within the message for each session. This allows the server to distinguish fresh requests from retransmissions.

Server-side mechanism: The dispatcher maintains a request history cache, which is a two-dimensional map that is keyed firstly by the client's IP address and port, and secondly by the request ID. Before dispatching a request to the handler, it will parse the message header and extract both keys. It will check the cache for a match:

1. If a match is found, the request is a retransmission, and we return the cached reply immediately.
2. Else, the handler executes and caches the reply bytes before returning them.

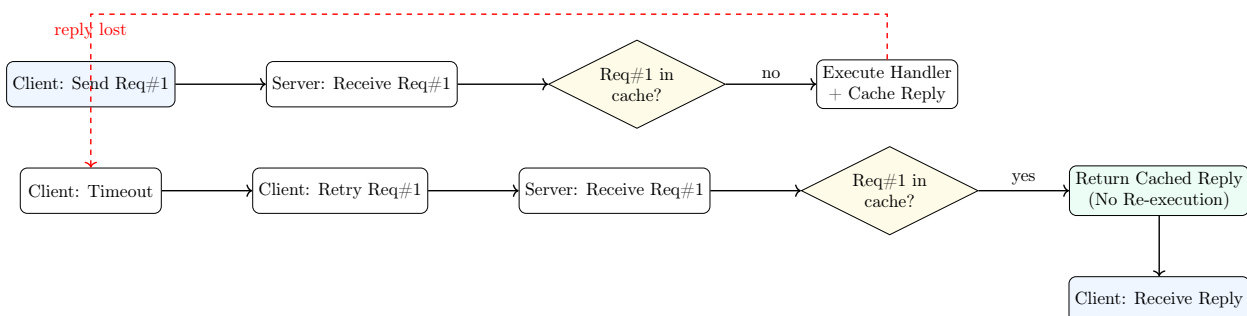


Figure 9: At-most-once semantics: duplicate requests are detected and cached replies are returned without re-execution.

Each request is processed a maximum of once. This is the ideal for banking, where monetary transfers must execute exactly once per submission as intended. However, this semantic requires the system to scale its memory usage proportionally to the number of active clients and their request backlog, which could grow greatly in production. Additionally, the use of an incremented request ID in the client introduces coordination overhead in this implementation.

To address these issues, when the server receives a new request with an ID of N , the dispatcher evicts previous cache entries with an ID below $(N - \text{windowSize})$, bounding cache memory to $O(\text{clients} * 10)$. This assumes

that clients never retransmit requests that are older than a certain time range. At the same time, all cache entries for the client are evicted when the client disconnects to save memory.

5.3 Loss Simulator Design

To validate the difference between both semantics when messages are lost, a loss simulator was implemented in the server. It utilises a seeded pseudo-random number generator, configured with a probability between 0.0 and 1.0.

The loss simulator can be injected at either of two points:

1. **Request-level:** After the server's UDP socket receives a datagram, the loss simulator may discard it before it is parsed. This forces the client to timeout and retransmit.
2. **Reply-level loss:** After the handler produces a reply, the loss simulator may drop it before it is sent to the client, simulating the scenario where the client's receive mechanism times out.

For this implementation, we focus on reply-level loss only, for simplicity of testing the results of different invocation semantics.

5.4 Experimental Validation

To validate the semantic models, a suite of 38 integration tests was developed. These tests launch the real C++ client and Go server as separate processes, communicating over localhost UDP. Key test scenarios include:

Table 5: Representative Fault Tolerance Test Cases

Operation	Semantics	Loss Rate	Expected Outcome	Observed
GetBalance	at-least-once	0%	Correct balance returned	True
GetBalance	at-least-once	30%	Correct balance as it is idempotent and re-tries are safe	True
Deposit \$100	at-least-once	0%	Balance +\$100, account notified	True
Deposit \$100	at-least-once	30%	Balance +\$100 to +\$200 upon packet loss and retry success	True
Deposit \$100	at-most-once	30%	Balance +\$100 exactly upon packet loss and duplicate detection	True
Transfer \$50	at-least-once	30%	Source -\$50 to -\$100 upon packet loss and retry success	True
Transfer \$50	at-most-once	30%	Source -\$50 exactly upon packet loss and duplicate detection	True

5.5 Results & Discussion

The integration test suite confirms the theoretical predictions.

Firstly, for idempotent operations like GetBalance, the correct balance is returned under both semantics and all loss rates. The operation is inherently safe to retry because it does not modify any states. Thus, at-least-once is acceptable for GetBalance.

Secondly, for non-idempotent operations (Deposit, Withdraw, Transfer), we analyse at-least-once semantics. When simulated packet loss occurs, duplicate requests cause multiple executions, which is unsafe for non-idempotent banking operations. For example:

- Transfer \$50 with 30% loss rate: If a single response packet is lost, the source account is debited twice, causing a \$100 total decrease instead of \$50.

For non-idempotent operations under at-most-once semantics, the server's request history cache detects retransmissions and prevents duplicate executions, which is safe for banking operations. In this scenario, we can validate that the response is returned from the cache:

- Transfer \$50 with 30% loss rate: Even if multiple response packets were lost, source is debited exactly \$50 and destination is credited exactly \$50.

To conclude, the choice of invocation semantics directly impacts correctness of the system. For banking systems that require this accuracy, and handles non-idempotent operations, at-most-once semantics are necessary.

6 Testing Strategy

The project employs a two-tier testing approach: unit tests for individual components with mocked dependencies, and end-to-end integration tests exercising the full system.

6.1 Unit Tests (C++ Client)

The client includes 48 unit tests across four GoogleTest suites, with dependency mocks to isolate each layer.

Table 6: C++ Unit Test Coverage Summary

Suite	Scope	Key checks
CommandEncoder	TLV request/response encoding for all services	Big-endian fields, variable-length strings, and IEEE-754 amount round-trip correctness
MessageSerializer	18-byte message header and payload framing	Header round-trip (IPv4, port, status, length), short-packet rejection, overflow-safe length handling
UDPSocket	UDP send/receive integration on loopback	Timeout options (<code>SO_RCVTIMEO</code> , <code>SO_SNDTIMEO</code>), <code>SO_REUSEADDR</code> , and <code>getsockname()</code> correctness
BankClient	CLI parsing and command construction	Input validation (account/currency/password) and correct request assembly before dispatch

By mocking components like `BaseSocket` and `BaseCommandEncoder`, the tests does not rely on network dependencies, and execute deterministically.

6.2 Integration Tests (End-to-End)

The integration test suite (`integration_test.go`) is written in Go and runs 38 test cases that pass on both Windows and Linux. Each test:

1. Spawns the real Go server with specified flags (e.g., ‘`-semantics=at-most-once -loss-rate=0.3`’).
2. Spawns the real C++ client, instructing it to connect to the server’s UDP address.
3. Executes a sequence of operations by injecting CLI commands via pipes.
4. Validates outcomes by querying the server’s in-memory state or comparing received replies.
5. Terminates both processes and reports pass/fail.

Table 7: Integration Test Categories and Coverage

Category	Coverage summary
Baseline (no loss)	Verifies all seven services end-to-end under normal network conditions.
Concurrency	Runs multiple clients in parallel to validate consistency and account-total conservation.
Fault tolerance (10%/30%/50%)	Compares at-least-once vs at-most-once behavior under packet loss, and confirms that duplicate side effects only occur in at-least-once for non-idempotent operations.
Stress and edge cases	Tests rapid-fire requests, large balances, and invalid boundary inputs (e.g., withdraw \$0).
Isolation	Executes each test in an isolated process/port with fresh server state to avoid cross-test interference.

7 Assumptions, Limitations & Future Work

7.1 System Assumptions

The implementation operates under the following assumptions:

1. **Single currency per account:** Each account is fixed to one currency (SGD, USD, or EUR), and transfers between accounts of different currencies are rejected.
2. **In-memory storage:** All account data is stored in a transient Go map, with no persistence to a disk or database. If the server crashes, all data is lost.

7.2 Known Limitations

The system can be improved in areas of memory management and security:

1. **Unbounded reply history growth (at-most-once):** The server's request history cache grows with each unique request, which could exhaust server memory in the long run. While the current implementation supports manual eviction, a time-based or size-based eviction policy could be used instead.
2. **No wire encryption:** All messages are transmitted insecurely in plaintext over UDP. An attacker can eavesdrop on account numbers, balances, and passwords, or forge replies.
3. **Fixed password length (8 bytes):** The wire protocol enforces an 8-byte password field, truncating longer passwords, and zero-padding shorter passwords. This is inflexible and weak for real passwords.

7.3 Future Work

Based on acknowledged assumptions and limitations, several extensions would be valuable:

1. **Persistent storage:** Implement a persistent SQL database for the account store, and use transactions to ensure atomicity of multi-account operations.
2. **Authentication and encryption:** Implement password hashing, add client-side TLS handshake for mutual authentication, and encrypt all message payloads using AES-256.
3. **Multi-currency and exchange rates:** Allow transfers between accounts in different currencies by utilising a live exchange rate API.

7.4 Key Takeaways

This project demonstrates several core principles of distributed systems design:

1. **Unreliable networks require explicit protocols:** UDP provides no guarantees, therefore custom retransmission and duplicate detection logic were essential for reliability.
2. **Invocation semantics shape correctness:** At-least-once and at-most-once have different guarantees. While idempotent operations are safe under at-least-once mode; non-idempotent operations require at-most-once. Choosing the wrong semantics silently corrupts data.
3. **Cross-language interoperability is possible but fragile:** Agreeing on byte-level formats (big-endian, IEEE-754, fixed-size fields) and marshalling every field explicitly is tedious but necessary for systems without a shared RPC framework.

8 Conclusion

This report presented a complete implementation of a distributed banking system that supports cross-language interoperability, custom binary protocol design, concurrent access control, and fault-tolerant invocation semantics.

It also successfully demonstrated core banking operations (Open, Close, Deposit, Withdraw, Transfer, GetBalance) with full atomic semantics and real-time monitoring. The infrastructure is supported by customisations

at the transport layer that use a custom 18-byte header format and Type-Length-Value payload encoding, negotiated without RPC frameworks.

The server's concurrency model combines a read-write mutex on the global account map with per-account mutexes for individual operations. Transfer operations use deterministic lock ordering to prevent deadlock while maintaining atomicity across two accounts.

Fault tolerance of the system is analysed through the evaluation of different invocation semantics, under simulated packet losses. While at-least-once mode has a simpler implementation, it leads to phantom duplicate executions on non-idempotent operations. Conversely, while at-most-once mode is safer for all operation types, where the request history cache ensures that each operation executes at most once regardless of network conditions, its implementation is more complex in terms of memory usage.

The comprehensive test suite with 48 unit tests and 38 integration tests validates correctness across the entire architecture, from low-level socket I/O to high-level banking logic, and from no packet loss to 50% loss scenarios.

This implementation serves as a blueprint for real-world distributed systems. It emphasises explicit protocol design, careful concurrency control, and empirical validation under realistic fault scenarios. While production banking systems would require additional security and memory management features, this project captures the essential challenges and solutions of distributed financial transactions.